WEEK 1
Connecting to Google Gemini

1. Project Overview

The objective of Week 1 is to build a first Generative AI application in Python. The program loads a Gemini API key from a .env file, connects to Google Gemini using the Google Gen AI SDK, sends a prompt to the gemini-3.5-flash-lite model, and prints the generated response in the terminal.

2. Objectives

- Set up a Python project with a virtual environment and install its dependencies.
- Create a Gemini API key in Google AI Studio.
- Load API credentials securely from a .env file instead of hard-coding them.
- Connect to Google Gemini using the Google Gen AI Python SDK.
- Send a prompt to a Gemini model and display the generated response.

3. Tools and technologies

Tool / Library	Purpose
Python 3	Programming language (project environment uses Python 3.14)
PyCharm	Code editor and integrated terminal
Google AI Studio	Used to create the Gemini API key
Google Gemini API	Cloud-based LLM provider
google-genai	Python SDK for Gemini
python-dotenv	Loads environment variables from .env

4. Project Folder Structure

The project folder is named Generative Ai Skill - 1. It contains the following files and folders:

```
Generative Ai Skill - 1/
├── .venv/              # Python virtual environment
├── .env                # Gemini API key (kept private)
├── main.py             # Program entry point
└── requirements.txt    # Python dependencies
```

PyCharm also creates an .idea folder for its project settings. It does not need to be edited.

Figure 1. Project folder and files in PyCharm.

5. Environment Setup on Windows

1. Open the project folder in PyCharm.
2. Open the integrated terminal (View > Tool Windows > Terminal).
3. Create a virtual environment (if it has not already been created): `python -m venv .venv`
4. Activate it: `.venv\Scripts\activate`. The terminal should show (.venv).
5. Install dependencies: `pip install -r requirements.txt`
6. Create a Gemini API key in Google AI Studio and save it in the .env file (see the next section).

6. Configuration: .env

The Gemini API key is stored in a .env file so that it is not hard-coded into the Python source. Replace the placeholder below with the API key created for your Google AI Studio project. The .env file must never be uploaded to GitHub or shown in screenshots.

```
GEMINI_API_KEY=YOUR_GEMINI_API_KEY
```

7. Dependencies: requirements.txt

Install all listed packages from the project terminal with:

```
google-genai
python-dotenv

pip install -r requirements.txt
```

8. Source Code

8.1 main.py

The program loads the .env file, reads the GEMINI_API_KEY value, and stops with an error message if the key is missing. It then creates a Gemini client, sends the prompt “What is Generative AI? Explain in simple words.” to the gemini-3.5-flash-lite model, and prints the text of the response.

```
import os
from dotenv import load_dotenv
```

```
from google import genai

# Load API key
load_dotenv()

api_key = os.getenv("GEMINI_API_KEY")

if not api_key:
    raise ValueError("Gemini API key not found!")

# Connect to Gemini
client = genai.Client(api_key=api_key)

# Send prompt
response = client.models.generate_content(
    model="gemini-3.5-flash-lite",
    contents="What is Generative AI? Explain in simple words."
)

# Display response
print("\nGemini Response:")
print(response.text)
```

9. Running the Application

Run these commands in the PyCharm terminal from inside the project folder:

```
.venv\Scripts\activate
python main.py
```

The program prints “Gemini Response:” followed by the generated explanation of Generative AI.

[Insert screenshot: terminal showing main.py running and the Gemini response]

Figure 2. Running main.py and receiving a Gemini response.

10. Result

A first Generative AI application was implemented in Python. It loads the Gemini API key securely from the .env file, connects to Google Gemini through the Google Gen AI SDK, sends a prompt to the gemini-3.5-flash-lite model, and prints the generated answer in the terminal.

11. Conclusion

This project introduces the basic workflow for calling a cloud-based LLM: configure credentials, create a client, send a prompt, and read the response. It is the foundation for the following weeks, where the same idea is organized into a provider-agnostic client with monitoring.